B24 Convolutional networks

Convolution from scratch, shapes, pooling, image tensors and augmentation, transfer learning and image-classifier evaluation.

B24-L1 Convolution, a small filter slid across an image

A convolution slides one small set of shared weights across the image. It is local, it works the same everywhere, and training learns its weights.

```python
def correlate(image, kernel):
    kh, kw = kernel.shape
    out = np.zeros((image.shape[0] - kh + 1, image.shape[1] - kw + 1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = (image[i:i + kh, j:j + kw] * kernel).sum()
    return out
```

What a convolution computes	At each position, the patch times the kernel, element by element, summed. The outputs form a feature map.
Output size without padding	(h - kh + 1, w - kw + 1).
Parameters in nn.Conv2d(in, out, k)	out * in * k * k weights plus out biases. Independent of image size.
Does nn.Conv2d flip the kernel?	No. It computes cross-correlation, the plain sliding sum.

B24-L2 Padding, stride, channels and the (N, C, H, W) layout

Conv2d reads (N, C, H, W). Output size is (H + 2p - k) // s + 1. Pad to keep the size, stride to shrink it, and permute images that arrive channels last.

Conv2d output size	(H + 2 * padding - kernel_size) // stride + 1, for height and width alike.
Keep the size with a k by k kernel	Stride 1 and padding (k - 1) / 2, or padding='same' (stride 1 only).
Conv2d input layout	(N, C, H, W). Convert (N, H, W, C) with x.permute(0, 3, 1, 2), never reshape.
Conv2d weight shape	(out_channels, in_channels, kH, kW).

B24-L3 Pooling, flattening and a complete small CNN

Conv, ReLU and pool blocks shrink the maps and grow the channels. Size the dense layer from the shape rule, or end with global average pooling so the size never matters.

```python
model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),     # 8x8 to 4x4
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),    # 4x4 to 2x2
    nn.Flatten(),                                                    # (N, 32 * 2 * 2)
    nn.Linear(32 * 2 * 2, 10),                                       # one logit per digit
)
```

What MaxPool2d(2) does	Largest value per non-overlapping 2 by 2 block. Halves height and width, rounding down. No weights.
Linear size after Flatten	channels * H * W of the last feature map. Work it out with the size rule, layer by layer.

Make a CNN accept any image size	End the convolutions with nn.AdaptiveAvgPool2d(1), then Flatten and Linear(channels, classes).
Typical CNN block	Conv2d (padding to keep the size), ReLU, MaxPool2d(2). Each block halves the map and grows the channels.

B24-L4 Images as tensors, normalisation and augmentation

Channels first, float32 in [0, 1], normalised with training statistics. Augment training images only, with changes that keep the label true.

```
train_tf = v2.Compose([v2.ToImage(), v2.RandomCrop(32, padding=4),
  v2.RandomHorizontalFlip(),
                       v2.ToDtype(torch.float32, scale=True), v2.Normalize(MEAN,
  STD)])
eval_tf = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True),
  v2.Normalize(MEAN, STD)])
```

Image to model input	(H, W, C) uint8 to (C, H, W) float32 in [0, 1], then normalise per channel with training statistics.
torchvision v2 steps for a uint8 image	v2.ToImage(), v2.ToDtype(torch.float32, scale=True), v2.Normalize(mean, std).
Where augmentation goes	Training transform only, applied in `__getitem__` so each epoch draws new changes. Validation and test get the fixed steps.
Choosing augmentations	Only changes that keep the label true: shifts and left-right flips for photos, never flips or half-turns for digits.

B24-L5 Transfer learning, a pretrained backbone with a new head

Load pretrained weights, freeze them, replace the head, build the optimiser afterwards from trainable parameters, and preprocess exactly as the backbone was trained.

```
weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, n_classes)
optimizer = torch.optim.AdamW((p for p in model.parameters() if p.requires_grad),
  lr=1e-3)
preprocess = weights.transforms()
```

Transfer learning recipe	Load pretrained weights, freeze them, replace the head, build the optimiser afterwards from trainable parameters.
ResNet-18 head	model.fc is Linear(512, 1000). Replace it with nn.Linear(model.fc.in_features, n_classes).
Preprocessing for a pretrained model	weights.transforms(): resize 256, centre crop 224, ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225].
Frozen batch norm in train mode	Its running statistics still update. Keep the frozen backbone in eval mode while the head trains.

B24-L6 Evaluating image classifiers, per class and on honest splits

Predict in eval mode, read recall per class from the confusion matrix, look at the errors, and split by the unit that makes near-duplicates.

```
cm = confusion_matrix([0, 0, 1, 1, 2, 2], [0, 1, 1, 1, 2, 0])
# [[1, 1, 0], [0, 2, 0], [1, 0, 1]]
recall = cm.diagonal() / cm.sum(axis=1)     # [0.5, 1.0, 0.5]
```

Class predictions from a classifier	model.eval(), torch.no_grad(), then logits.argmax(dim=1).
Recall per class from a confusion matrix	Diagonal over row totals (rows are true classes). Guard empty rows: NaN, not 0.
Near-duplicate images	Bursts, repeat shots and saved augmentations. Split by shelf, store or session, never by photo.
What to report for an image classifier	Accuracy with an interval, recall for every class, the confusion matrix, and a look at the worst errors.